ChatGPT

Implementuj platební bránu Stripe v rezervačním systé

Požadavky:
1. Stripe Checkout Session — po vytvoření rezervace p
Stripe platební stránku
2. Webhook endpoint (POST /api/webhooks/stripe) —
událostí:
- checkout.session.completed → aktualizace stavu r
"Paid"
- checkout.session.expired → zrušení rezervace
3. Propojení s existujícím BookingEntity — přidat Paym
StripeSessionId, StripePaymentIntentId
4. Konfigurace: Stripe API keys v appsettings.json (seł
Stripe:SecretKey, Stripe:WebhookSecret)
5. Frontend: tlačítko "Zaplatit" u nepaid rezervace → r
Stripe Checkout
6. Webhook signature verification (ověření, že request
Stripe)

Výstup: funkční platební flow od vytvoření rezervace p
platby.

Níže najdete návrh implementace platebního toku pomocí Stripe Checkout do stávajícího systému. Postup vychází z průzkumu zdrojového kódu repozitáře `demo-projekt` a ze S dokumentace – server po vytvoření rezervace vytvoří Checkout session a zákazník je pře platební stránku Stripe docs.stripe.c , webhook následně ověří podpis události a aktualizuj podle typu události medium.co .

Backend (ASP.NET Core – projekt `PhysioBook`)

1. Přidání typů pro platby

Enum `PaymentStatus` – do adresáře `PhysioBook/Data/Enums` přidejte soubor `PaymentStatus.cs`:

C#

```
namespace PhysioBook.Data.Enums;

/// <summary>
/// Stav platby rezervace.
/// </summary>
public enum PaymentStatus
{
    /// Rezervace ještě nebyla zaplacena.
    Unpaid,

    /// Rezervace byla úspěšně zaplacena.
    Paid,

    /// Checkout session vypršela, rezervace je zrušena.
    Expired
}
}
```

Rozšíření `Booking` **entity** – ve `PhysioBook/Data/Entities/Booking.cs` přidejte

C#

```
public PaymentStatus PaymentStatus { get; set; } = PaymentStatus.Unpaid
public string? StripeSessionId { get; set; }
public string? StripePaymentIntentId { get; set; }
```

Do `BookingConfiguration` přidejte mapování (aby se enum ukládal jako řetězec):

C#

```
builder.Property(b => b.PaymentStatus)
       .HasConversion<string>()
       .IsRequired();
builder.Property(b => b.StripeSessionId);
builder.Property(b => b.StripePaymentIntentId);
```

Nastavení Stripe – vytvořte třídu `StripeSettings`:

C#

```
public class StripeSettings
{
    public string PublishableKey { get; set; } = string.Empty;
    public string SecretKey { get; set; } = string.Empty;
    public string WebhookSecret { get; set; } = string.Empty;
}
```

V `appsettings.json` doplňte:

JSON

```
"Stripe": {
  "PublishableKey": "pk_test_xxx",
  "SecretKey": "sk_test_xxx",
  "WebhookSecret": "whsec_xxx"
}
```

V `ServiceRegistrations.cs` načtěte nastavení a inicializujte Stripe:

C#

```
builder.Services.Configure<StripeSettings>(configuration.GetSection("St
var stripeSettings = configuration.GetSection("Stripe").Get<StripeSetti
Stripe.StripeConfiguration.ApiKey = stripeSettings.SecretKey;
```

Do souboru `PhysioBook.csproj` přidejte závislost na knihovně Stripe:

XML

```
<ItemGroup>
  …
  <PackageReference Include="Stripe.net" Version="45.1.0" />
</ItemGroup>
```

2. Rozšíření odpovědí a mapování

Rozšiřte záznamy `CreateBookingResponse`, `GetBookingByIdResponse` a `GetBookingsItemResponse` o vlastnost `PaymentStatus` (`string paymentStatus`
V mapovacích třídách (`CreateBookingMappings`, `GetBookingByIdMappings`, `GetBookingsMappings`) mapujte `booking.PaymentStatus.ToString()` na nový p
Při vytváření rezervace (`CreateBookingMappings.ToEntity`) inicializujte `Payment` `PaymentStatus.Unpaid`.

3. Endpoint pro vytvoření Stripe Checkout session

Přidejte statickou třídu `PayBookingEndpoint` např. do `PhysioBook/Features/Payment` ji v `MapBookingEndpoints()`:

C#

```
// Vytvoření Stripe Session
var stripeSettings = config.GetSection("Stripe").Get<Stripe
var options = new Stripe.Checkout.SessionCreateOptions
{
    Mode = "payment",
    PaymentMethodTypes = new List<string> { "card" },
    LineItems = new List<Stripe.Checkout.SessionLineItemOpt
    {
        new()
        {
            Quantity = 1,
            PriceData = new Stripe.Checkout.SessionLineItem
            {
                Currency = "czk",
                UnitAmount = (long)(booking.Service.Price *
                ProductData = new Stripe.Checkout.SessionLi
                {
                    Name = booking.Service.Name
                }
            }
        }
    },
```

V `EndpointRegistration` pak doplňte:

C#

```
bookingsGroup.MapPayBookingEndpoint();
```

4. Webhook pro zpracování událostí

Vytvořte třídu `StripeWebhookEndpoint` např. v `PhysioBook/Features/Payments`:

C#

```
            try
            {
                stripeEvent = Stripe.EventUtility.ConstructEvent(
                    json, sigHeader, stripeSettings.WebhookSecret);
            }
            catch (StripeException)
            {
                return Results.BadRequest();
            }

            if (stripeEvent.Type == "checkout.session.completed")
            {
                var session = stripeEvent.Data.Object as Stripe.Checkou
                var booking = await db.Bookings
                    .FirstOrDefaultAsync(b => b.StripeSessionId == sess
                if (booking != null)
                {
                    booking.PaymentStatus = PaymentStatus.Paid;
                    booking.Status = BookingStatus.Confirmed;
                    await db.SaveChangesAsync();
                }
            }
```

Zavolejte `app.MapStripeWebhookEndpoint()` z `Program.cs` či `EndpointRegistrati` `/api/bookings` group). Webhooky ověřují podpis pomocí tajného klíče z konfigurace a požadavky medium.co .

Frontend (`physiobook-frontend`)

1. Typy a služby

Enum a typ rezervace – v `src/features/bookings/types.ts` přidejte:

TypeScript

```
export enum PaymentStatus {
  Unpaid = "Unpaid",
  Paid = "Paid",
  Expired = "Expired",
}

export interface Booking {
  // …
  status: string;
  paymentStatus: PaymentStatus;
  stripeSessionId?: string;
  stripePaymentIntentId?: string;
}
```

Booking service – v `src/features/bookings/bookingService.ts` přidejte funkci rezervace:

TypeScript

```
export async function payBooking(id: string): Promise<{ sessionId: stri
  const res = await axios.post(`/api/bookings/${id}/pay`);
  return res.data;
}
```

Hook – doplňte `usePayBooking` do `src/features/bookings/hooks.ts` :

TypeScript

```
export function usePayBooking() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => payBooking(id),
    onSuccess: () => {
      queryClient.invalidateQueries(["bookings"]);
    },
  });
}
```

2. Tlačítko „Zaplatit“

Na stránce „Moje rezervace“ (`src/pages/MyBookingsPage.tsx`) zobrazte tlačítko jen p
rezervace:

TypeScript

```
{booking.paymentStatus === PaymentStatus.Unpaid && (
  <Button onClick={() => handlePay(booking.id)}>Zaplatit</Button>
)}
```

Funkce `handlePay` :

TypeScript

```
const { mutateAsync: payBooking } = usePayBooking();
const stripePromise = loadStripe(import.meta.env.VITE_STRIPE_PUBLISHABL

async function handlePay(bookingId: string) {
  const { sessionId, url } = await payBooking(bookingId);
  // Varianta A — přesměrování pomocí sessionId (bez odhaleného URL):
  const stripe = await stripePromise;
  if (stripe) await stripe.redirectToCheckout({ sessionId });
  // Varianta B — rovnou otevřít URL:
  // window.location.href = url;
```

3. Migrace databáze

Po úpravách entit spusťte migra
`StripePaymentIntentId`) vytv

Bash

```
dotnet ef migrations add
dotnet ef database updat
```

Shrnutí

Navržené změny rozšiřují systén
a přidávají možnost zaplatit reze
vytvoří Checkout session a vrátí
docs.stripe.c . Webhook ověřuje
nastaví platbu na „Paid“, zatímc

Tím vznikne plnohodnotný plate

oprav chyby:
Backend:

/src/PhysioBook/Features/Payments/StripeWebhookEr
8): error CS8072: An expression tree lambda may not

/src/PhysioBook/Features/Payments/StripeWebhookEr
8): error CS8072: An expression tree lambda may not
propagating operator.
[/src/PhysioBook/PhysioBook.csproj]
/src/PhysioBook/Features/Payments/PayBookingEndp
error CS0029: Cannot implicitly convert type 'long' to
'System.DateTime?'
[/src/PhysioBook/PhysioBook.csproj]
/src/PhysioBook/Features/Payments/PayBookingEndp
error CS1503: Argument 2: cannot convert from
'System.Threading.CancellationToken' to
'Stripe.RequestOptions' [/src/PhysioBook/PhysioBoo

Frontend:
src/features/bookings/bookingService.ts(5,21): error
Cannot find name 'axios'.
src/features/bookings/hooks.ts(117,23): error TS2552

name 'useQueryClient'. Did you mean 'queryClient'?
src/features/bookings/hooks.ts(118,10): error TS2304
name 'useMutation'.
src/features/bookings/hooks.ts(119,33): error TS2552
name 'payBooking'. Did you mean 'usePayBooking'?
src/pages/AdminBookingsPage.tsx(186,25): error TS7
implicitly has an 'any' type because expression of type
be used to index type
'Record<BookingStatus, { label: string; className: st
No index signature with a parameter of type 'string'
type 'Record<BookingStatus, { label: string; className
src/pages/AdminBookingsPage.tsx(209,40): error TS
Argument of type 'string' is not assignable to paramet
'BookingStatus'.
src/pages/MyBookingsPage.tsx(33,39): error TS2552
name 'usePayBooking'. Did you mean 'payBooking'?
src/pages/MyBookingsPage.tsx(34,25): error TS2304
name 'loadStripe'.
src/pages/MyBookingsPage.tsx(34,48): error TS2339
'env' does not exist on type 'ImportMeta'.
src/pages/MyBookingsPage.tsx(132,32): error TS705
implicitly has an 'any' type because expression of type
be used to index type
'Record<BookingStatus, { label: string; className: st
No index signature with a parameter of type 'string'
type 'Record<BookingStatus, { label: string; className
src/pages/MyBookingsPage.tsx(188,54): error TS230
name 'PaymentStatus'.
src/pages/MyBookingsPage.tsx(189,26): error TS230
name 'Button'.
src/pages/MyBookingsPage.tsx(189,85): error TS230
name 'Button'.
src/pages/MyBookingsPage.tsx(191,34): error TS234
type 'string' is not assignable to parameter of type 'Bo

Z uvedeného seznamu chyb je patrné, že některé změny v předchozí úpravě způsobily k
Následující kroky tyto chyby opravují:

Backend (C#)

Null-propagace v expression tree – v `StripeWebhookEndpoint` nepoužívejte `ses` uvnitř LINQ. Před vyhodnocením výrazu si hodnotu uložte do pomocné proměnné:

C#

```
// místo: FirstOrDefaultAsync(b => b.StripeSessionId == session?.Id)
var sessionId = session != null ? session.Id : null;
var booking = await db.Bookings.FirstOrDefaultAsync(b => b.StripeSessio
```

Totéž použijte při získávání rezervace podle `session.PaymentIntentId`.

Typ vlastnosti `ExpiresAt` – knihovna Stripe ve verzi v projektu očekává `DateTime` `long`. Nastavte expiraci na:

C#

```
ExpiresAt = DateTime.UtcNow.AddMinutes(30)
```

Volání `SessionService.CreateAsync` – přetížení metody nepřijímá `Cancellatior` parametr. Můžete jej předat prostřednictvím `RequestOptions`, případně vynechat:

C#

```
var session = await service.CreateAsync(options);
// nebo s tokenem:
var session = await service.CreateAsync(options, new Stripe.RequestOpti
```

Ostatní drobnosti – ujistěte se, že vlastnost `PaymentStatus` je mapována přes `HasConversion<string>()`, aby se enum správně ukládal do databáze.

Frontend (TypeScript/React)

Správný klient API – všechny HTTP požadavky používají instanci `api` z `src/shared/BaseApiService.ts`. V souboru `bookingService.ts` definujte funkc takto:

TypeScript

```
import api from '@/shared/BaseApiService';

// …

async function payBooking(id: string): Promise<{ sessionId: string; url
  const res = await api.post(`/bookings/${id}/pay`);
  return res.data;
}

export const bookingService = {
  …,
  payBooking, // přidejte tuto funkci do exportu
};
```

Reaktní query hooks – v `hooks.ts` naimportujte `useQueryClient` a `useMutatio` `@tanstack/react-query` a `payBooking` z vašeho service:

TypeScript

```
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { bookingService } from './bookingService';

export function usePayBooking() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: bookingService.payBooking,
    onSuccess: () => queryClient.invalidateQueries(['bookings']),
  });
}
```

Načtení Stripe – v `MyBookingsPage.tsx` přidejte potřebné importy a opravte přístu `import.meta.env`:

TypeScript

```
import { usePayBooking } from '@/features/bookings/hooks';
import { PaymentStatus } from '@/features/bookings/types';
import { loadStripe } from '@stripe/stripe-js';
// pokud používáte vlastní tlačítko:
import Button from '@/components/Button'; // nebo nahraďte <Button> kla

// …
const stripePromise = loadStripe((import.meta as any).env.VITE_STRIPE_P
```

Použití hooku ve stránkách – v `MyBookingsPage.tsx` je třeba importovat a použít (namísto dříve chybně importovaného `payBooking`) a volat `payBooking.mutateAs

TypeScript

```
const { mutateAsync: payBooking } = usePayBooking();
async function handlePay(bookingId: string) {
  const { sessionId } = await payBooking(bookingId);
  const stripe = await stripePromise;
  await stripe?.redirectToCheckout({ sessionId });
}
```

Chybějící typové importy – chyby „Cannot find name 'PaymentStatus' / 'Button'" vy importů `PaymentStatus` a komponenty tlačítka. Pokud vlastní komponentu `Buttor` použijte přímo `<button>` s potřebnými třídami.

Indexování podle statusu – v `AdminBookingsPage.tsx` a `MyBookingsPage.tsx` p `statusConfig` typu `Record<BookingStatus, …>`, ale `bookingItem.status` je ty Přidejte explicitní přetypování:

TypeScript

```
const status = statusConfig[bookingItem.status as BookingStatus] || sta
```

Totéž u `STATUS_OPTIONS`, kde můžete hodnoty definovat jako `BookingStatus | 'all'`

Po provedení výše uvedených úprav by se projekty měly bez problémů přeložit a funkce fungovat správně.

backend: src/features/bookings/hooks.ts(2,45): error Cannot find module '@tanstack/react-query' or its cor type declarations.
src/pages/MyBookingsPage.tsx(5,28): error TS2307: module '@stripe/stripe-js' or its corresponding type d

Chyby hlásily, že knihovny `@tanstack/react-query` a `@stripe/stripe-js` nejsou sou Namísto jejich zavádění můžete funkci pro zaplacení rezervace realizovat pouze pomocí přesměrování přes URL, což odstraní závislosti na zmíněných balíčcích. Postup:

1. Úpravy `bookingService.ts`

Doplňte do vašeho service metodu `payBooking`, která využije stávající instanci `api`:

TypeScript

```
import api from '@/shared/BaseApiService';
// …

async function payBooking(id: string): Promise<{ sessionId: string; url
  const response = await api.post(`/bookings/${id}/pay`);
  return response.data;
}

export const bookingService = {
  …,
  payBooking,
};
```

2. Jednoduchý hook `usePayBooking` bez `@tanstack/react-query`

V souboru `src/features/bookings/hooks.ts` lze původní hook nahradit vlastním:

TypeScript

```
import { useState, useCallback } from 'react';
import { bookingService } from './bookingService';

export function usePayBooking() {
  const [loading, setLoading] = useState(false);

  const payBooking = useCallback(async (id: string) => {
    setLoading(true);
    try {
      return await bookingService.payBooking(id);
    } finally {
      setLoading(false);
    }
  }, []);

  return { payBooking, loading };
}
```

Tím odpadá potřeba importovat `useQueryClient` nebo `useMutation`.

3. Úprava `MyBookingsPage.tsx`

Odstraňte importy z `@stripe/stripe-js` a místo toho využijte přesměrování na URL, k

TypeScript

```
import { usePayBooking } from '@/features/bookings/hooks';
import { PaymentStatus } from '@/features/bookings/types';
// není třeba importovat loadStripe ani stripe-js

export default function MyBookingsPage() {
  // …
  const { payBooking, loading: paying } = usePayBooking();

  async function handlePay(bookingId: string) {
    const { url } = await payBooking(bookingId);
    // přímo přesměrování na platební stránku
    window.location.href = url;
  }

  // zbytek komponenty zůstává
}
```

4. Ostatní stránky

Pokud používáte informace o stavu platby také v dalších komponentách (např. `AdminBookingsPage.tsx`), není potřeba je měnit; stále pracují pouze se stavem rezerva zmíněných úpravách by už projekt neměl vyžadovat balíčky `@tanstack/react-query` a `@stripe/stripe-js`, a tedy se vyřeší chybové hlášky.

Tato řešení vychází z předpokladu, že klientskou aplikaci stačí po úspěšné inicializaci Str

přesměrovat na URL, kterou vygeneruje server – toto je plně podporovaný a dokumento
Stripe, kde server vytvoří Checkout session a klient se na ni přesměruje docs.stripe.c .

Zdroje

src/features/bookings/hooks.ts(121,52): error TS2559:
has no properties in common with type
'InvalidateQueryFilters<readonly unknown[]>'.
src/pages/MyBookingsPage.tsx(40,19): error TS2339
'redirectToCheckout' does not exist on type 'Stripe'.

Po analýze zdrojových souborů v repozitáři jsem zjistil, že zbývající chyby v TypeScriptu neupravenými částmi kódu, které stále spoléhají na balíčky `@tanstack/react-query` a `@stripe/stripe-js`. Tyto knihovny ale nejsou v projektu nainstalované, což vedlo k c find module…" a následně k typu chybějícímu `redirectToCheckout`. Navrhuji proto upra byl čistý, bez závislosti na těchto balíčcích.

Úprava `physiobook-frontend/src/features/bookings/hooks.ts`

Odstraňte importy z `@tanstack/react-query`:

TypeScript

```
// původní
import { useMutation, useQueryClient } from '@tanstack/react-query'
// nový stav — tyto importy lze kompletně smazat
```

Nahraďte implementaci `usePayBooking` vlastní implementací bez React Query:

TypeScript

```
// původní funkce
export function usePayBooking() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: bookingService.payBooking,
    onSuccess: () => queryClient.invalidateQueries(['bookings']),
  });
}
```

TypeScript

```
// nová implementace
export function usePayBooking() {
  const [loading, setLoading] = useState(false);

  const payBooking = useCallback(async (id: string) => {
    setLoading(true);
    try {
      return await bookingService.payBooking(id);
    } finally {
      setLoading(false);
    }
  }, []);

  return { payBooking, loading };
}
```

Tato verze nespoléhá na React Query a využívá pouze React `useState` a `useCall`
stavu načítání. V případě potřeby lze s daty po úspěšné platbě pracovat ručně (např
`refetch` v jiném hooku).

Úprava `physiobook-frontend/src/pages/MyBookingsPage.tsx`

Odstraňte import a použití `@stripe/stripe-js`:

TypeScript

```
// původní
import { loadStripe } from '@stripe/stripe-js';
...
const stripePromise = loadStripe((import.meta as any).env.VITE_STRI
```

Odstraňte tyto řádky, protože balík není k dispozici a přesměrování lze udělat přímo
Upravte použití `usePayBooking`:

TypeScript

```
// původní
const { mutateAsync: payBooking } = usePayBooking();
...
const { sessionId } = await payBooking(bookingId);
const stripe = await stripePromise;
await stripe?.redirectToCheckout({ sessionId });
```

Změňte na:

TypeScript

```
const { payBooking, loading: paying } = usePayBooking();

async function handlePay(bookingId: string) {
  const { url } = await payBooking(bookingId);
  window.location.href = url; // přesměrování na Stripe Checkout
}
```

Tato změna volá serverovou endpointu `/bookings/{id}/pay`, obdrží `url` v odpov
přesměruje uživatele na platební stránku Stripe; balík `stripe-js` tak není potřeba
Aktualizujte destrukturování `usePayBooking`: Místo `mutateAsync` destrukturova
pracujte s novými vracenými hodnotami `payBooking` a `loading` (případně přejme
`paying`, aby nedocházelo ke kolizi se stavem načítání rezervací).

Po těchto úpravách:

Kód již nebude používat chybějící balíčky `@tanstack/react-query` a `@stripe/str`
Chyba „Type 'string[]' has no properties in common with type 'InvalidateQueryFilters
protože hook `usePayBooking` už nepracuje s `invalidateQueries`.

Zdroje

funkce `redirectToCheckout` nepoužívate a přesměrování probíhá čistým způsobe

Implementací výše uvedených změn by se frontend měl přeložit bez chyb a platební tok

Webhook nefunguje. Platba přes Stripe checkout proj
mě zpět na payment-success, ale stripe listen ukazuje
event. V logách backendu nic. Webhook secret mám v
appsettings.json stejný jako co mi vypsal stripe listen.

Chybový status 400 při zpracování webhooků většinou znamená, že server přijal požada
ho ověřit. Podle Stripe dokumentace se tato situace obvykle stává z těchto důvodů:

- **Chybný endpoint secret** – pro ověření podpisu musíte použít přesně ten podpisový
  poskytuje pro danou endpointu. Dokumentace připomíná, že tajný klíč z Dashboardu
  který generuje CLI; nesprávný klíč vede k chybě „Webhook signature verification fail
- **Nesprávně načtené tělo požadavku** – Stripe vyžaduje „raw" tělo požadavku v UTF-
  frameworky tělo upraví (např. JSON deserializace), čímž podpis přestane sedět docs

Vzhledem k tomu, že ve vašem logu není vidět žádný záznam, pravděpodobně nedojde a
handleru nebo selže ověření podpisu. Níže jsou návrhy na kontrolu a úpravu kódu:

1. Ověřte endpoint secret

Ujistěte se, že `Stripe:WebhookSecret` v `appsettings.json` odpovídá aktuální hodnot
`listen` vypisuje tajný klíč začínající `whsec_…`). Sekrety z Dashboardu a CLI nejsou zam
docs.stripe.c .

2. Povolte webhook bez autentizace

Webhooky by neměly vyžadovat JWT ani jinou autentizaci, jinak Stripe obdrží odpověď 4
mapování endpointu přidejte `AllowAnonymous()` :

C#

```
app.MapPost("/api/webhooks/stripe", HandleWebhook)
    .AllowAnonymous();
```

3. Čtěte neupravené tělo požadavku

Middleware může tělo upravit nebo vyčíst. Před čtením povolte bufferování a resetujte p
`ConstructEvent` dostal originální JSON:

C#

```
app.MapPost("/api/webhooks/stripe", async (HttpContext context, PhysioB
{
    context.Request.EnableBuffering(); // dovolí vícenásobné čtení

    using var reader = new StreamReader(context.Request.Body, Encoding.
    var body = await reader.ReadToEndAsync();
    context.Request.Body.Position = 0;

    var signature = context.Request.Headers["Stripe-Signature"].ToStrin
    var secret = options.Value.WebhookSecret;

    Stripe.Event stripeEvent;
    try
    {
        // tolerance 300, throwOnApiVersionMismatch: false
        stripeEvent = Stripe.EventUtility.ConstructEvent(body, signatur
    }
    catch (StripeException)
    {
        return Results.BadRequest();
    }
```

Používáte-li atribut `.Accepts<string>("application/json")` nebo `[FromBody]`, ASP
mohl tělo předem deserializovat a tím změnit podpis; tyto atributy v případě webhooku o

docs.stripe.c

Zdroje

C#

```
logger.LogInformation("R
```

Po zaplacení mě Stripe přesměruje na /payment-succe
stránka je prázdná

Tím ověříte, že požadavek dorazil a kde případně selhává.

5. Ověřte správnou

Důvod, proč je stránka `/payment-success` prázdná, je prostý: v současném frontendu n
cestu vůbec definována žádná komponenta ani route. Klient po zaplacení přijde na URL,
nastavili jako `SuccessUrl` (`http://localhost:5173/payment-success?session_id=
{{CHECKOUT_SESSION_ID}}` docs.stripe.c), ale router v `App.tsx` takovou stránku nezná
nenajde, co zobrazit.

Po těchto úpravách by měla verifikace podpisu probíhat korektně a Stripe CLI by měla hl
received (200)“. Pokud i nadále dostáváte 400, zkontrolujte v logu, zda se nevyvolá `Str
zprávou „No signatures found“ – pak je třeba znovu ověřit tajný klíč a způsob čtení těla

Jak problém vyřešit

- **Vytvořte komponentu PaymentSuccessPage** (např. `physiobook-frontend/src/pages/PaymentSuccessPage.tsx`). Tato stránka může uživateli pod
  zaplacení a nabídnout mu návrat na přehled rezervací. Příklad implementace:

TypeScript

```
export default function PaymentSuccessPage() {
  const [params] = useSearchParams();
  const sessionId = params.get('session_id'); // pokud chcete dále p

  return (
    <Layout>
      <div className="max-w-xl mx-auto text-center py-16">
        <h1 className="text-3xl font-bold mb-4">Platba úspěšná</h1>
        <p className="text-gray-700 mb-6">
          Děkujeme za vaši platbu. Vaše rezervace byla úspěšně zapla
        </p>
        <Link
          to="/my-bookings"
          className="inline-flex items-center px-6 py-3 bg-teal-600
        >
          Zobrazit moje rezervace
        </Link>
      </div>
    </Layout>
  );
}
```

Podobně můžete vytvořit i `PaymentCancelPage.tsx` pro `CancelUrl`.

Zaregistrujte nové stránky v routeru: V souboru `physiobook-frontend/src/App.` importy a route pro obě stránky:

TypeScript

```
const PaymentSuccessPage = lazy(() => import('./pages/PaymentSuccess
const PaymentCancelPage = lazy(() => import('./pages/PaymentCancelPa

// …v rámci <Routes>…
<Route path="/payment-success" element={<ProtectedRoute><PaymentSuc
<Route path="/payment-cancel" element={<ProtectedRoute><PaymentCanc
```

Opravte hook `usePayBooking` a přesměrování: Pokud jste odstranili balíček `@str` nepoužívejte `loadStripe().redirectToCheckout`. Implementujte jednoduchý hoc serverové API a pak přesměruje prohlížeč na URL vrácenou serverem:

TypeScript

```
// src/features/bookings/hooks.ts
export function usePayBooking() {
  const [loading, setLoading] = useState(false);

  const payBooking = useCallback(async (id: string): Promise<{ sess
    setLoading(true);
    try {
      return await bookingService.payBooking(id);
    } finally {
      setLoading(false);
    }
  }, []);

  return { payBooking, loading };
}
```

A v `MyBookingsPage.tsx` upravte `handlePay` takto:

TypeScript

```
const { payBooking } = usePayBooking();

async function handlePay(bookingId: string) {
  const { url } = await payBooking(bookingId);
  window.location.href = url; // přesměruje uživatele na Stripe Che
}
```

Tím se zbavíte závislosti na `@tanstack/react-query` a `@stripe/stripe-js`.

Ověřte konfiguraci serveru: Ujistěte se, že v backendu je `SuccessUrl` a `CancelUr`

nastaveno na nové cesty. Váš server vrací URL sessionu a redirect po zaplacení se c
`/payment-success?session_id=…` , což pak router zpracuje.

Po vytvoření stránky a route pro `/payment-success` nebude uživatel po zaplacení konč
stránce, ale uvidí potvrzení a může se vrátit do svých rezervací.

src/App.tsx(21,45): error TS2307: Cannot find module
'./pages/PaymentCancelPage' or its corresponding typ
chybí modul

Chyba `TS2307: Cannot find module './pages/PaymentCancelPage'` znamená, že R
načíst komponentu `PaymentCancelPage` , která však v projektu neexistuje nebo není na
Proto TypeScript nemůže najít ani její typové deklarace.

Jak problém vyřešit:

1. **Vytvořte soubor** `physiobook-frontend/src/pages/PaymentCancelPage.tsx` (pc
stránku pro zrušenou platbu používat). Může vypadat podobně jako úspěšná stránk

TypeScript

```
const [params] = useSearchParams();
const bookingId = params.get('booking_id');

return (
  <Layout>
    <div className="max-w-xl mx-auto text-center py-16">
      <h1 className="text-3xl font-bold mb-4">Platba zrušena</h1>
      <p className="text-gray-700 mb-6">
        Platba byla zrušena nebo vypršela. Vaše rezervace (ID: {bo
      </p>
      <Link
        to="/my-bookings"
        className="inline-flex items-center px-6 py-3 bg-teal-600
      >
        Zobrazit moje rezervace
      </Link>
    </div>
  </Layout>
);
}
```

Tento soubor uložte do `src/pages` , aby odpovídal importu `./pages/PaymentCanc`
v `App.tsx` .

2. **Zajistěte registraci stránky v routeru**, aby se na ni uživatel dostal, například v `App`

TypeScript

```
const PaymentSuccessPage = lazy(() => import('./pages/PaymentSuccess
const PaymentCancelPage  = lazy(() => import('./pages/PaymentCancelP

// …
<Route path="/payment-success" element={<ProtectedRoute><PaymentSuc
<Route path="/payment-cancel"  element={<ProtectedRoute><PaymentCan
```

3. **Pokud stránku pro zrušenou platbu nepotřebujete**, jednoduše odstraňte řádek imp
route z `App.tsx` . Pak nebude React očekávat, že takový modul existuje.

Ujistěte se, že název souboru a cesta přesně odpovídají tomu, co importujete
(`./pages/PaymentCancelPage`). Jakmile tento soubor vytvoříte nebo odstraníte import
kompilaci zmizí.